

Tool Exploration: `edgar-crawler` for SEC Filing Extraction

Marco Zhang

February 2026

1 Introduction

Our goal: download SEC filings in bulk from the authoritative source (SEC EDGAR) and extract structured, item-level text—not raw HTML blobs—for use across multiple research projects. Three requirements shape our tool choice.

Batch Downloading

SEC EDGAR hosts millions of filings. Downloading them one at a time is impractical:

```
# Manual, one at a time
download("AAPL", "10-K", "2023")
download("AAPL", "10-K", "2022")
download("AAPL", "10-K", "2021")
# ... repeat 1000 times for all companies/years
```

We need a tool that automates bulk download:

```
# Automated bulk download
download_all(
  years=[1996, 2024],
  forms=["10-K", "10-Q"],
  companies="ALL" # or specific list
)
# Downloads 100,000+ filings automatically
```

Multi-Filter Support

Some tools (e.g., `py-edgar`) allow filtering by only one dimension at a time—CIK or form type, but not both. We need to combine filters: specific companies + date range + form type + quarter:

```
# Single filter (py-edgar):
download(cik="0000320193") # Only Apple
# But can't combine: "Apple + Microsoft, 2020-2023, 10-Ks only"

# Multiple filters (edgar-crawler):
download(
    tickers=["AAPL", "MSFT", "GOOGL"],
    years=[2020, 2023],
    forms=["10-K"],
    quarters=[4]
)
```

Structured Extraction

Most tools return raw HTML—one giant blob per filing. For research, we need item-level text (e.g., Item 1A Risk Factors, Item 7 MD&A) already separated:

```
# Without section extraction: one giant blob
<!DOCTYPE html><html><head>...
ITEM 1. BUSINESS...50 pages...
ITEM 1A. RISK FACTORS...80 pages...
[Embedded tables, HTML tags, images, etc.]
```

```
# With section extraction: structured JSON
{"item_1": "...", "item_1A": "...", "item_7": "..."}

```

Only two tools satisfy all three requirements: **edgar-crawler** (open-source, Python) and **sec-api.io** (paid web service). Since **sec-api.io** is a paid service, we explore **edgar-crawler** specifically below.

2 How We Use It

Each project simply uses its own `config.json` pointing to its own folders. The tool is stateless—all state lives in the config and output directories:

```
project-A/config.json -> project-A/RAW_FILINGS/ -> project-A/
    EXTRACTED_FILINGS/
project-B/config.json -> project-B/RAW_FILINGS/ -> project-B/
    EXTRACTED_FILINGS/
```

No code changes needed between projects—just swap the config. Each `config.json` specifies two parts corresponding to the two-step workflow:

Step 1: Download Raw Filings (`download_filings`)

Specify which filings to fetch—company tickers, year range, form types, and quarters—then run `python download_filings.py`. This downloads raw HTML/txt files from EDGAR and produces a metadata CSV.

```
"download_filings": {
    "start_year": 2023,
    "end_year": 2024,
    "quarters": [1, 2, 3, 4],
    "filing_types": ["10-K", "10-Q"],
    "cik_tickers": ["AAPL"],           // or [] for all companies
    "user_agent": "Your Name (your@email.edu)",
    "raw_filings_folder": "RAW_FILINGS",
    "indices_folder": "INDICES",
    "filings_metadata_file": "FILINGS_METADATA.csv",
    "skip_present_indices": true
}
```

Step 2: Extract Items (`extract_items`)

Parse each raw filing into a structured JSON with one key per item. Run `python extract_items.py`.

```
"extract_items": {
    "raw_filings_folder": "RAW_FILINGS",
    "extracted_filings_folder": "EXTRACTED_FILINGS",
```

```

    "filings_metadata_file": "FILINGS_METADATA.csv",
    "filing_types": ["10-K", "10-Q"],
    "items_to_extract": [],           // [] = all items
    "remove_tables": true,
    "skip_extracted_filings": true
}

```

Output: EXTRACTED_FILINGS/ contains one JSON per filing. For a 10-K:

```

{"item_1": "Apple Inc. designs...",
 "item_1A": "The following discussion of risk factors...",
 "item_7": "This MD&A section...", ...}

```

For a 10-Q:

```

{"part_1": "(full Part 1 blob)",
 "part_1_item_2": "Management's Discussion and Analysis...",
 "part_2": "(full Part 2 blob)",
 "part_2_item_1": "Legal Proceedings...", ...}

```

3 Configuration & Limitations

The tool's authors provide extensive checks for 10-K, 10-Q, and 8-K extraction, which produce good structured results. Everything sounds perfect—and overall, it is reliable. But some limitations emerge from how the configuration works. To understand them, we look at the two config stages: file-level (Step 1) and item-level (Step 2). We skip all other config options; see the full reference at <https://github.com/leftterisloukas/edgar-crawler>.

3.1 File-Level Extraction (filing_types)

The `filing_types` list controls which form types are downloaded. The default is `["10-K", "8-K", "10-Q"]`. Importantly, these three are the only types the authors have verified with multiple checks—the GitHub README states: “We currently support the structured JSON output for 10-K, 10-Q and 8-K documents.” This raises a few things to be aware of.

EDGAR uses exact string matching on form types (`df.Type.isin(filing_types)`). Specifying just `"10-K"` misses all variants. We fetched the real EDGAR index for Q1+Q2

2005 (589,394 total filings) and confirmed: `.isin(["10-K"])` captures only 7,726 out of 14,643 annual-report-related filings—**missing 47%**. Nine distinct variants exist:

Type	Count	Description
10-K	7,726	Standard annual report
10KSB	2,561	Small business annual (discontinued 2009)
NT 10-K	2,040	Late filing notification
10-K/A	1,540	Amended annual report
10KSB/A	687	Amended small business annual
NTN 10K	49	Non-timely notification
NT 10-K/A	31	Late filing notification for amendment
10-KT	7	Transition period annual
10-KT/A	2	Amended transition period

Table 1: Annual-report-related filing types in EDGAR Q1+Q2 2005 index.

All 9 variants are fetchable (HTTP 200 confirmed). Must explicitly list all desired variants in the config. If we want consistent extraction from any of these, we need to further understand their document structure and potentially update the extraction code.

3.2 Item-Level Extraction (`extract_items.py`)

The tool maintains a static list of all items to extract. For 10-K, this is `item_list_10k`:

```
["1", "1A", "1B", "1C", "2", "3", "4", "5", "6", "7", "7A", "8",
 "9", "9A", "9B", "9C", "10", "11", "12", "13", "14", "15", "16"]
```

3.2.1 Stale Item Config

This list is a superset of all known 10-K items across all eras—but it requires manual updates whenever the SEC introduces, eliminates, or repurposes an item. Two recent examples: Item 1C (Cybersecurity) was mandated for fiscal years ending after Dec 15, 2023; Item 6 (Selected Financial Data) was eliminated in 2021, though companies still include it as “[RESERVED]”.

We ran the tool’s own test suite and all 3 tests failed. The primary cause: the author added Item 1C to `item_list_10k` but didn’t regenerate the test fixtures, so every expected JSON was missing the `item_1C` key. We raised a PR on the original repo to fix the stale fixtures. The deeper lesson: the item set requires active management as SEC rules evolve.

To understand what to expect from the extraction, we built an item evolution table from the 62 10-K test fixtures (1994–2023):

Item	First Appears	Notes
1–6, 8–14	1994 (earliest)	Consistently present
7A	1997	Quantitative risk disclosures
1A (Risk Factors)	~2004	SOX-era; empty pre-2004
9A (Controls)	~2004	SOX-era; empty pre-2004
9B (Other Info)	~2004–2006	Appears with SOX items
1B (Staff Comments)	~2006	Spotty early on
15 (Exhibits)	1994	Consistent
16 (10-K Summary)	Rarely	Empty in nearly all fixtures
1C (Cybersecurity)	2023+	SEC mandate FY ending Dec 15, 2023+
9C (Foreign Jurisd.)	~2020+	HFCAA
6 (Consolidated Data)	1994–2023	Still present despite SEC elimination 2021

Table 2: 10-K item evolution across 62 test fixtures (1994–2023).

Three eras emerge. **Pre-SOX (1994–2003)**—no 1A/9A/9B; risk factor research is impossible at item level. **Post-SOX (2004–2022)**—full set except 1C/9C/16; the “standard” era for most research. **Modern (2023+)**—1C added, 6 eliminated; structural breaks for longitudinal studies. Key takeaway: empty items pre-2004 are mostly regulatory non-existence, not extraction failure.

3.2.2 Combined Items Detection

Companies sometimes combine items in their filing (e.g., “Items 9, 9A, and 9B”). When this happens, all content is assigned to the first item only; the subsequent items appear empty. The tool does not detect or flag this. Needs post-processing detection for affected items.

3.2.3 10-Q Time-Series Consistency (Part-Level vs. Item-Level)

The tool extracts 10-Q at two levels: the whole part blob (`part_1`, `part_2`) and individual items within each part (`part_1_item_1`, `part_1_item_2`, etc.). For 6 out of 200 (3%) test fixtures—all from 1993–1995—the tool successfully grabs the entire Part 1 text (6K–14K chars), but when it tries to split into individual items, it gets nothing: `part_1_item_1` through `part_1_item_4` are all empty. Part 2 items extract correctly in all cases.

The cause: old `.txt` filings (pre-~1996) don’t have explicit “ITEM 1”, “ITEM 2” headers within Part I—they go straight from “PART I - FINANCIAL INFORMATION” into the content. The regex finds nothing. In the real-world EDGAR population (not curated test fixtures), the failure rate is likely higher than 3%. If you need item-level 10-Q data (e.g., MD&A = Part 1 Item 2) from pre-~2000 filings, fall back to the `part_1` blob and

parse it yourself.

3.3 Spot Check

We manually verified 5 filings across eras from test fixture expected JSONs, plus one modern 10-Q:

Filing	Era	Format	Extracted	Notes
Turner Broadcasting 1993 10-K	Pre-2000	.txt	14/22	1A/7A/9A empty (pre-SOX).
Internet America 2003 10-K	Early HTML	.htm	17/22	Has 9A. Missing 1A/1B.
Walt Disney 2012 10-K	Post-SOX	.htm	19/22	1A = 28K, 7 = 75K chars.
FedEx 2023 10-K	Modern	.htm	22/22	All items. Item 6 = “[RESERVED]”.
Zurn Industries 1993 10-Q	Pre-2000	.txt	Part 1: failed	Blob = 11K but item keys empty.
Microsoft 2024 10-Q	Modern	.htm	Full success	Part 1: 4 items, Part 2: 5 items.

Table 3: Spot-check results across filing eras.

Interpretation: (1) 10-K extraction is reliable across all eras. (2) 10-Q has a pre-2000 Part 1 failure mode. (3) Modern extraction quality is excellent.

Summary

#	Finding	Impact	Ref.
1	10-Q Part 1 item extraction fails on old .txt filings	High	§3.2.3
2	<code>filing_types</code> exact matching excludes 47% of variants	High	§3.1
3	Item structural breaks across three decades	High	§3.2.1
4	Item set requires manual tracking as SEC changes rules	Med-High	§3.2.1
5	Combined items not detected	Medium	§3.2.2
6	10-K extraction reliable across all eras (1993–2024)	Positive	§3.3
7	Modern extraction (2010+) quality is excellent	Positive	§3.3

Table 4: Summary of findings.

4 Other Minor Things & Next Steps

4.1 Minor Things

- **Table removal impact on MD&A (Item 7).** Tables embedded in narrative sections like MD&A are removed by default (`remove_tables: true`). This is fine for pure text analysis but can be toggled off if tables are needed.

- **10-Q part separation fragility.** The two-part separation uses 4 heuristic bug-detection cases. Failure modes include ToC contamination and arbitrary thresholds. Known limitation; can't do better for now.
- **Fiscal year alignment.** The `quarters` config filters by which calendar quarter the filing was *submitted* to EDGAR, not the period it reports on. Companies have different fiscal year-ends (e.g., Apple = September, most others = December), so the same calendar quarter contains filings for different reporting periods. Always filter by `period_of_report` in the metadata for cross-sectional comparisons.
- **Survivorship bias.** Could happen if downloading only a subset of companies via ticker lists (tickers of delisted firms may not resolve). Avoided by using CIK identifiers or downloading the full universe (`cik_tickers: []`).

4.2 Next Steps

If we decide to go with this tool:

1. Set up a multi-project environment with clear hierarchy and markdown instruction files that feed Claude Code for automated use (preliminary hierarchy already drafted). Since each project uses its own `config.json` (Section 2), different projects can specify different filing types, item sets, and time ranges as needed—e.g., one project downloads only 10-K, another only 10-Q.
2. Encode the findings from this report as project-level instructions for the Claude agent.